Reinforcement Learning

1. Robots → walk
2. self driving cars
3. Go, Chess games



→ trial & error

win

maximize rewards

Agent → Environment

Rewards



Reinforcement Learning

APNA COLLEGE

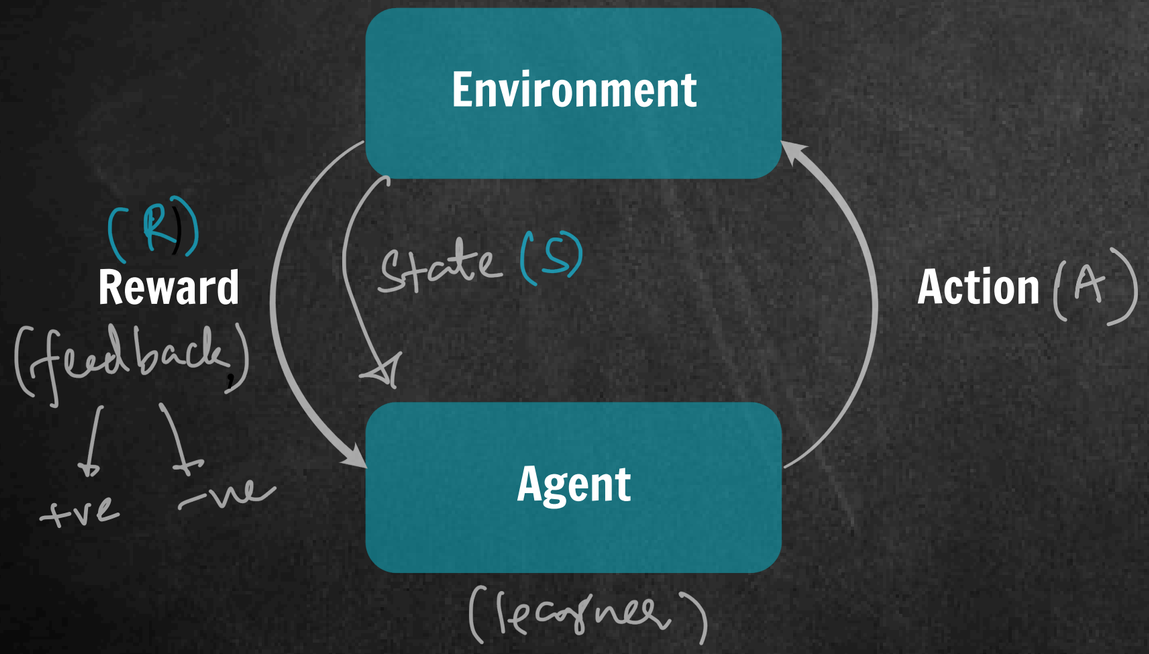
Autonomous

1. 2013 — DeepMind → Deep Q-Network
   - RL + DL
   - integrate

2. 2016 — Go
   - AlphaGo → Lee Sedol

Reinforcement Learning

Components



ultimate goal
↓
maximize Rewards

S → snapshot of env

1. $S_t, A_t \longrightarrow R_{t+1}$

2. Policy (π) → strategy choosing an Action

3. (G_t) Return (Total Future Rewards)
Gain

APNA COLLEGE

Reinforcement Learning

Components



Reinforcement Learning

Components

$\gamma = 0$ immediate returns

$\gamma = 1$ long term

1. Action (A_t)
2. Rewards (R_t)
3. State (S_t)
4. Policy (π) $S \rightarrow A$ $\pi\left(\frac{a}{s}\right)$
5. Returns (G_t)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

$\gamma \rightarrow$ discount factor

$$0 \leq \gamma \leq 1$$

Reinforcement Learning

Components

APNA COLLEGE

(6) Value function (VF)

↓

Returns estimate based on a particular state S.

(a) State VF → $V_\pi(s)$ estimate return based on state S followed by policy π

(b) Action VF → $Q_\pi(a, s)$

- optimal state VF → $V_*(s)$ → max possible return in all policies

- optimal Action VF → $Q_*(a, s)$

Reinforcement Learning

Markov DecisionProcess(MDP) — math model

↓

sequence of State, Action, Reward .....

$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3 \ldots\ldots$

- Continuous → stock trading algos
- terminate → chess / Go → Episode

$[S_0, A_0, R_1, S_1, A_1, R_2 \ldots$ (A_0 → piece move; S_1 → ↓)

APNA COLLEGE

Reinforcement Learning

Markov DecisionProcess(MDP)

MARKOV Property rule

idea: next state ← curr state, curr action
not the past history

$S_{t+1} \leftarrow S_t, A_t$

limitations

1. not all situation/env follow MDP
2. Reward → single val

APNA COLLEGE

Reinforcement Learning

Markov DecisionProcess(MDP)

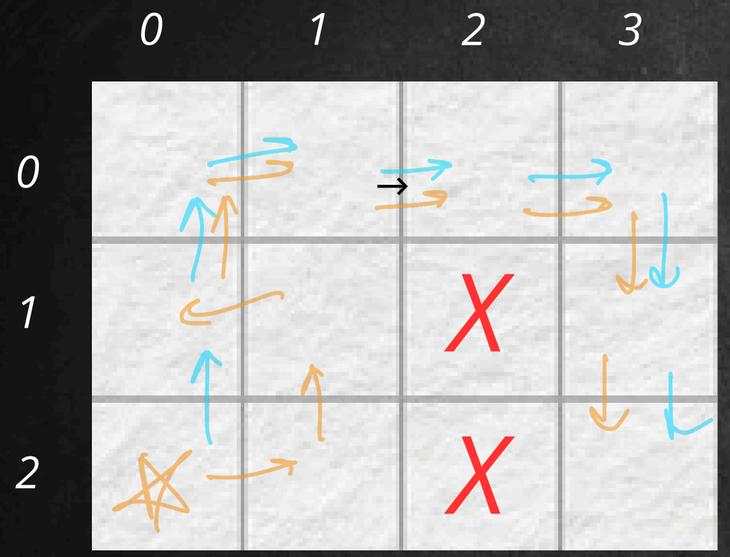


1. Q-learning
2. SARSA
3. Policy Gradients

APNA COLLEGE

Reinforcement Learning

Grid Example



allowed moves ⇒ up, down, left, right
0 1 2 3

terminate ⇒ E reach OR 15 moves moves

MDP

S = cell (i, j) → 2 nums

A = 0-3 → 4 nums

R = target (E) → 0
non-E cells → -1

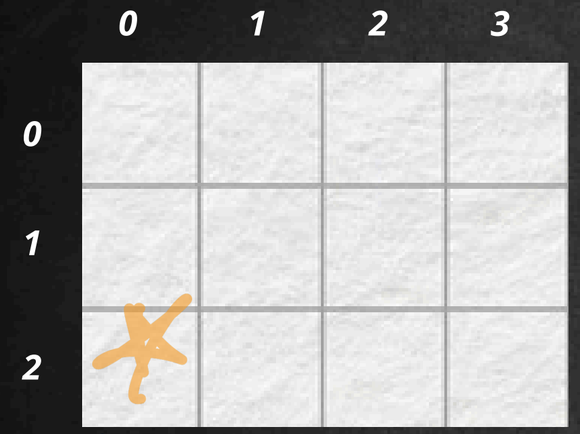
APNA COLLEGE

Reinforcement Learning

Policy optimization

APNA COLLEGE

Grid Example



World Model → understand how the env behaves

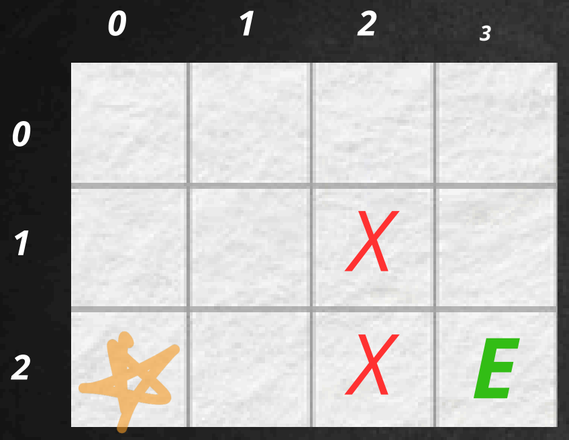
1. Model based RL
2. Model free RL

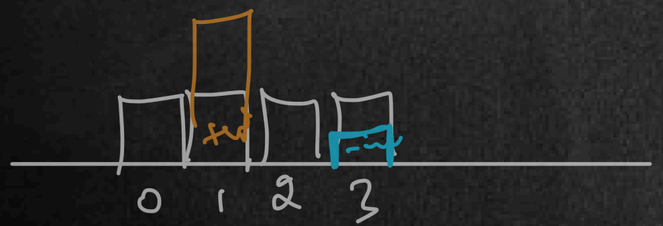
$$P\left(\frac{s_{t+1}, r_{t+1}}{s_t, a_t}\right)$$

-13
-15
-9
-

Reinforcement Learning

Grid Example





Policy (π)

Policy optimization

Policy → random (0, 1, 2, 3)

series of actions → better results / worse results

L1 r2 ...

Policy Gradient Method

actions → +ve

actions → -ve

State VF

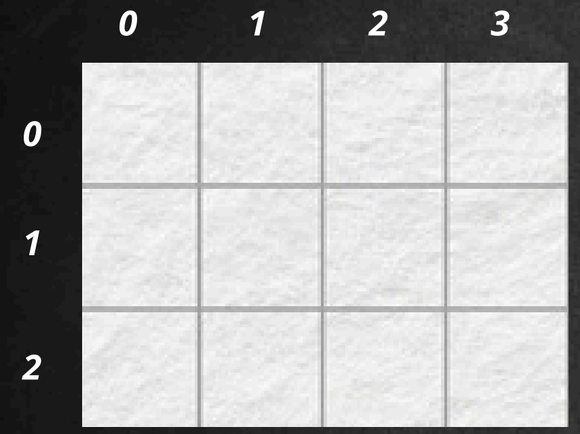
Action VF

APNA COLLEGE

Reinforcement Learning

Policy $S \rightarrow A$

State	Action
(2,0)	A_0
0,0	A_1
1,1	A_2
0,2	A_3
⋮	



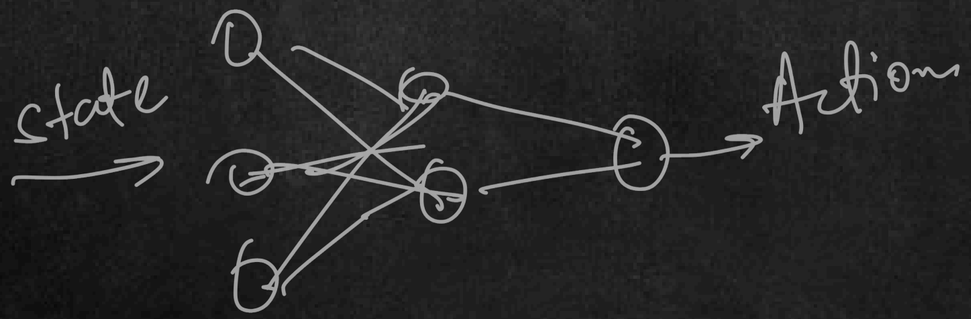
up → 0

down - 1

left → 2

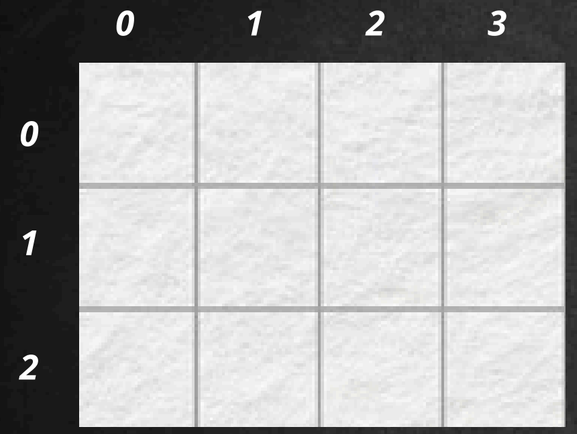
right → 3

neural networks



Reinforcement Learning

Grid Example



model free RL : Action-Value f > State-Value f

Q-function

$Q_\pi(s,a)$ → estimate the expected returns of S & A for π

$Q_*(s,a)$ → pick the highest estimated action

↓

Greedy

Policy

Q-learning

Deep Q-Networks

APNA COLLEGE

Reinforcement Learning

Exploration–Exploitation Tradeoff



ϵ-Greedy Strategy → Q learning

ϵ (probability to explore)

```
if(r < ϵ)
    explore
else
    exploit ⇒ optimal Q-val
```

① ϵ = 0.1 (10% chances to explore)

$0 \leq \epsilon \leq 1$

②

Action	Q-value
up	5
down	3
left	1
right	10

③ random value (0 to 1) → r

Reinforcement Learning

Exploration–Exploitation Tradeoff

start → explore ↑ exploit ↓

end → ε ↓ learning ability ↑

ε → gradually decay

↓

probab of explore